PROJECT REPORT

HOMEWORK 5

Written and Prepared by:

Roxanne Lutz

University of New Mexico

November 4, 2025

HANDWORKED PROBLEMS

See following pages for the handwritten, worked problems:

Prob 1 For distinct $x_0 \dots x_n$ points corresponding to real $y_0 \dots y_n$, there is a unique polynomial $P_n(x)$ of @most degree n such that $P_n(x_i) = y_i$ for $0 \le i \le n$.

We are able to assume the existance of this polynomial to begin.

Assume there is a polynomial $Q(x)$ such that $Q_n(x_i) = P_n(x_i)$ for $0 \le i \le n$. $Q(x)$ is also of at most degree n, but is distinct from $P_n(x)$.

Consider the polynomial $P_n(x) - Q_n(x)$ → This is also a polynomial of degree at most n. But, see: $P_n(x) - Q_n(x)$ ⟶ This is essentially yielding a new function which has roots at the intersection points of $P_n(x)$ & $Q_n(x)$, which are guaranteed to be at least $x_0 \dots x_n$ → $n+1$ points/roots!

But, since $P_n(x) - Q_n(x)$ is at most degree n, it can have at most n roots. But since $P_n(x) - Q_n(x)$ intersect at a guaranteed $n+1$ points, we reach a contradiction, since $P_n(x) - Q_n(x)$ cannot have $n+1$ roots.

Therefore, $P_n(x)$ must equal $Q_n(x)$, or $P_n(x) - Q_n(x) = 0$. $P_n(x) = Q_n(x)$, and $P_n(x)$ must be unique given $x_0 \dots x_n$.

Prob 2 | Lagrange Interp for poly of least degree w/

x	0	2	3	4
y	7	11	28	63

= 4 points.

0 1 2 3

* Poly numerical will be at most degree 4 − 1 = degree 3.

* Form: $P_3(x) = \sum_{i=0}^{3} l_i(x) f(x_i)$

$= l_0(x) f(x_0) + l_1(x) f(x_1) + l_2(x) f(x_2) + l_3(x) f(x_3)$.

* Where $l_k(x) = \prod_{\substack{j \neq k \\ j=0}}^{n} \left(\frac{x - x_j}{x_k - x_j} \right)$ for $0 \leq k \leq n$.

$$l_0(x) = \frac{(x-x_1)(x-x_2)(x-x_3)}{(x_0-x_1)(x_0-x_2)(x_0-x_3)} = \frac{(x-2)(x-3)(x-4)}{(0-2)(0-3)(0-4)}$$

$(-2)(-3)(-4) = 6(-4) = -24$

$$= -\frac{1}{24}(x-2)(x-3)(x-4)$$

$$l_1(x) = \frac{(x-x_0)(x-x_2)(x-x_3)}{(x_1-x_0)(x_1-x_2)(x_1-x_3)} = \frac{(x-0)(x-3)(x-4)}{(2-0)(2-3)(2-4)}$$

$(2)(-1)(-2) = 4$.

$$= \frac{1}{4} x(x-3)(x-4).$$

$$l_2(x) = \frac{(x-x_0)(x-x_1)(x-x_3)}{(x_2-x_0)(x_2-x_1)(x_2-x_3)} = \frac{(x-0)(x-2)(x-4)}{(3-0)(3-2)(3-4)}$$

$(3)(1)(-1) = -3$.

$$= -\frac{1}{3} x(x-2)(x-4).$$

$$l_3(x) = \frac{(x-x_0)(x-x_1)(x-x_2)}{(x_3-x_0)(x_3-x_1)(x_3-x_2)} = \frac{(x-0)(x-2)(x-3)}{(4-0)(4-2)(4-3)}$$

$$(4)(2)(1) = 8.$$

$$= \frac{1}{8}x(x-2)(x-3).$$

Finally:

$$P_3(x) = -\frac{7}{24}(x-2)(x-3)(x-4) + \frac{11}{4}x(x-3)(x-4) - \frac{20}{3}x(x-2)(x-4) + \frac{63}{8}x(x-2)(x-3).$$

Prob 3 Use Newton / div differences to interp & compare w/ 2 on uniqueness.

x	0	2	3	4
y	7	11	28	63

x_n	$f(x_n)$	$f[,]$	$f[,,]$	$f[,,,]$
0	(7) a_0	$\frac{11-7}{2-0} = \frac{4}{2} = (2)$ a_1		
2	11		$\frac{17-2}{3-0} = \frac{15}{3} = (5)$ a_2	
3	28	$\frac{28-11}{3-2} = \frac{17}{1} = 17$		$\frac{9-5}{4-0} = (1)$ a_3
4	63	$\frac{63-28}{4-3} = 35$	$\frac{35-17}{4-2} = \frac{18}{2} = 9$	

Newton's interpolation:

$$P_3(x) = 7 + 2(x-0) + 5(x-0)(x-2) + 1(x-0)(x-2)(x-3).$$

$$\boxed{P_3(x) = 7 + 2x + 5x(x-2) + x(x-2)(x-3).} = P2_3(x).$$

Let's assume they are not equivalent (i.e., they are not unique, the 2 are distinct polynomials). So, we'll look at a simplified Lagrange to compare. ($P2_3(x) \stackrel{?}{=} P1_3(x)$)

The first Lagrange → let's distribute that out → let $= P1_3(x)$.

$$P1_3(x) = -7/24(x-2)(x-3)(x-4) + 11/4\,x(x-3)(x-4) - 28/3\,x(x-2)(x-4) + 63/8\,x(x-2)(x-3).$$

$$= -7/24[(x^2-5x+6)(x-4)] + 11/4[x(x^2-7x+12)] - 28/3[x(x^2-6x+8)] + 63/8[x(x^2-5x+6)].$$

$$= -7/24(x^3-4x^2-5x^2+20x+6x-24) + 11/4(x^3-7x^2+12x) - 28/3(x^3-6x^2+8x) + 63/8(x^3-5x^2+6x).$$

$$= -7/24(x^3-9x^2+26x-24) + 66/24(x^3-7x^2+12x) - 224/24(x^3-6x^2+8x) + 189/24(x^3-5x^2+6x).$$

$$= -\tfrac{7}{24}x^3 + \tfrac{63}{24}x^2 - \tfrac{182}{24}x + 7$$
$$+ \tfrac{66}{24}x^3 - \tfrac{462}{24}x^2 + \tfrac{792}{24}x$$
$$- \tfrac{224}{24}x^3 + \tfrac{1344}{24}x^2 - \tfrac{1792}{24}x$$
$$+ \tfrac{189}{24}x^3 - \tfrac{945}{24}x^2 + \tfrac{1134}{24}x.$$

$$\tfrac{24}{24}x^3 - \tfrac{48}{24}x^2 - \tfrac{48}{24}x + 7 = P1_3(x) = x^3 - 2x + 7.$$

→ Now, let's distribute Newton:

$$P2_3(x) = 7 + 2x + 5x(x-2) + (x)(x-2)(x-3)$$
$$= 7 + 2x + 5x^2 - 10x + (x^2 - 2x)(x-3)$$
$$= 7 + 2x + 5x^2 - 10x + x^3 - 3x^2 - 2x^2 + 6x$$
$$= x^3 - 2x + 7$$

So: $P1_3(x) \stackrel{?}{=} P2_3(x)$

$$x^3 - 2x + 7 \stackrel{?}{=} x^3 - 2x + 7$$
$$0 = 0$$ → The polynomials are equivalent! Therefore, they are not distinct, but the same unique polynomial in different forms.

Therefore, there is no contradiction of uniqueness theorem from our results. Although the polynomials look wildly different, they are in fact the same.

Prob 4 | Unique $P(x)$, deg $2 \geq$ s.t.
$P(0)=0$, $P(1)=1$, $P'(\alpha)=2$ for any pt in $\alpha \in [0,1]$
except for ONE (α_0) - i.e., there is an α_0 that
CANNOT be $P'(\alpha_0)=2$.

(1) What is α_0?

$P(x) = ax^2+bx+c$

→ Condition $P(0)=0$ implies c must be 0:

$0 = a(0^2)+b(0)+c$

$c = 0$.

→ Now left with:

$P(x) = ax^2+bx$.

→ Condition $P(1)=1$ now set up:

$1 = a+b$.

→ and let's find $P'(x)$ & find a contradictory value.

$P'(x) = 2ax+b$

$P'(\alpha_0) = 2a\alpha_0+b = 2$

$2 = 2a\alpha_0+b$, $1 = a+b$

$2 = a+b+1$.

$a+b+1 = 2a\alpha_0+b$

$a+1 = 2a\alpha_0$.

$1 = 2a\alpha_0 - a$.

$1 = a(2\alpha_0-1)$

$\frac{1}{2\alpha_0-1} = a$ → If $\alpha_0 = 1/2$, a is undefined. Therefore, our value for $\boxed{\alpha_0 = 1/2}$

→ Generally, using α, let's find the polynomial.

$$a = \frac{1}{2\alpha - 1}, \quad \alpha \neq 1/2, \quad \alpha \in [0,1], \quad c = 0$$

$$P(\alpha) = \frac{1}{2\alpha - 1}\alpha^2 + b\alpha$$

$$1 = a + b$$

$$1 = \frac{1}{2\alpha - 1} + b \rightarrow b = 1 - \frac{1}{2\alpha - 1} = \frac{2\alpha - 1 - 1}{2\alpha - 1} = \frac{2\alpha - 2}{2\alpha - 1}$$

So

$$\boxed{P(\alpha) = \frac{1}{2\alpha - 1}\alpha^2 + \frac{2\alpha - 2}{2\alpha - 1}\alpha, \quad \alpha \neq 1/2, \quad \alpha \in [0,1]}$$

Prob 5 | g interpolates f @ $x_0 \dots x_{n-1}$
h interpolates f @ $x_1 \dots x_n$.
Show following interpolates $x_0 \dots x_n$:

$$z(x) = g(x) + \frac{x_0 - x}{x_n - x_0}[g(x) - h(x)]$$

First, the 2 endpoints in interpolation that are not covered by both g & h are x_0 & x_n.
So, let's ensure that $z(x_0) = f(x_0)$ & $z(x_n) = f(x_n)$, thereby interpolating those 2 points:

① $z(x_0) = g(x_0) + \frac{x_0 - x_0}{x_n - x_0}[g(x_0) - h(x_0)]$ → $\frac{x_0 - x_0}{x_n - x_0} \to 0$

$z(x_0) = g(x_0) = f(x_0)$ → by definition, $g(x_0)$ must equal $f(x_0)$. ✓

② $z(x_n) = g(x_n) + \frac{x_0 - x_n}{x_n - x_0}[g(x_n) - h(x_n)]$

$= g(x_n) - \frac{x_n - x_0}{x_n - x_0}[g(x_n) - h(x_n)]$ → $\frac{x_n - x_0}{x_n - x_0} \to 1$

$= g(x_n) - g(x_n) + h(x_n)$

$z(x_n) = h(x_n) = f(x_n)$ → by definition, $h(x_n)$ must equal $f(x_n)$ ✓

And finally, for an arbitrary $i \in [1, n-1]$.

$$z(x_i) = g(x_i) + \frac{x_0 - x_i}{x_n - x_0}[g(x_i) - h(x_i)]$$

both of these interpolate over these values, thus

$$z(x_i) = f(x_i) + \frac{x_0 - x_i}{x_n - x_0}[f(x_i) - f(x_i)] \to 0$$

$z(x_i) = f(x_i)$ ✓ → Therefore, $z(x)$ interpolates overall $x_0 \dots x_n$.

Prob 6) trapezoidal rule for numerical integration results from approximating f by a first degree spline, then

$$\int_a^b f(x)\,dx \approx \int_a^b S(x)\,dx$$

On $[a,b]$, we will assume we are dealing with a spline (1st degree) that interpolates $(a, f(a))$ and $(b, f(b))$. The equation for the line will then be

$$S(x) - y_1 = \frac{y_2 - y_1}{x_2 - x_1}(x - x_1) \quad \left\{ \text{let } x_1 = a,\ x_2 = b,\ y_1 = f(a),\ y_2 = f(b) \right.$$

$$S(x) - f(a) = \frac{f(b) - f(a)}{b - a}(x - a)$$

$$S(x) = \frac{f(b) - f(a)}{b - a}(x - a) + f(a)$$

$$\text{let } K = \frac{f(b) - f(a)}{b - a}. \quad S(x) = Kx - aK + f(a)$$

$$\int_a^b S(x)\,dx = \int_a^b Kx\,dx - \int_a^b \underbrace{aK + f(a)}_{\text{constants}}\,dx$$

(Kx: K is a constant)

$$\int_a^b S(x)\,dx = K\frac{x^2}{2}\Big|_a^b - (aK + f(a))\,x\Big|_a^b$$

$$= K\left(\frac{b^2 - a^2}{2}\right) - (aK + f(a))(b - a)$$

$$= \frac{f(b) - f(a)}{b - a} \cdot \frac{(b - a)(b + a)}{2} - \left(a\frac{f(b) - f(a)}{b - a} + f(a)\right)^{(b-a)}$$

$$= \frac{f(b) - f(a)}{2}(b + a) - a(f(b) - f(a)) + f(a)(b - a)$$

$$= (f(b) - f(a))\left(\frac{b + a}{2} - a\right) + f(a)(b - a)$$

$$= (f(b) - f(a))\left(\frac{b - a}{2}\right) + f(a)(b - a)$$

$$= (b-a)\left[\frac{f(b)}{2} - \frac{f(a)}{2} + f(a)\right]$$

$$+ \frac{f(a)}{2}$$

$$= \frac{f(a)+f(b)}{2}(b-a)$$ ⟵ which is the trapezoidal rule for approximating $\int_a^b f(x)\,dx$.

You can use many first-degree splines at interval steps to sum many different integrals from arbitrary $[a, b]$ as well. But we will learn this as a single line for now.

COMPUTER/MATLAB PROBLEM 1

ORIGINAL PROBLEM STATEMENT

"Consider the Runge function given by:

$$f(x) = 1/(1 + x^2)$$

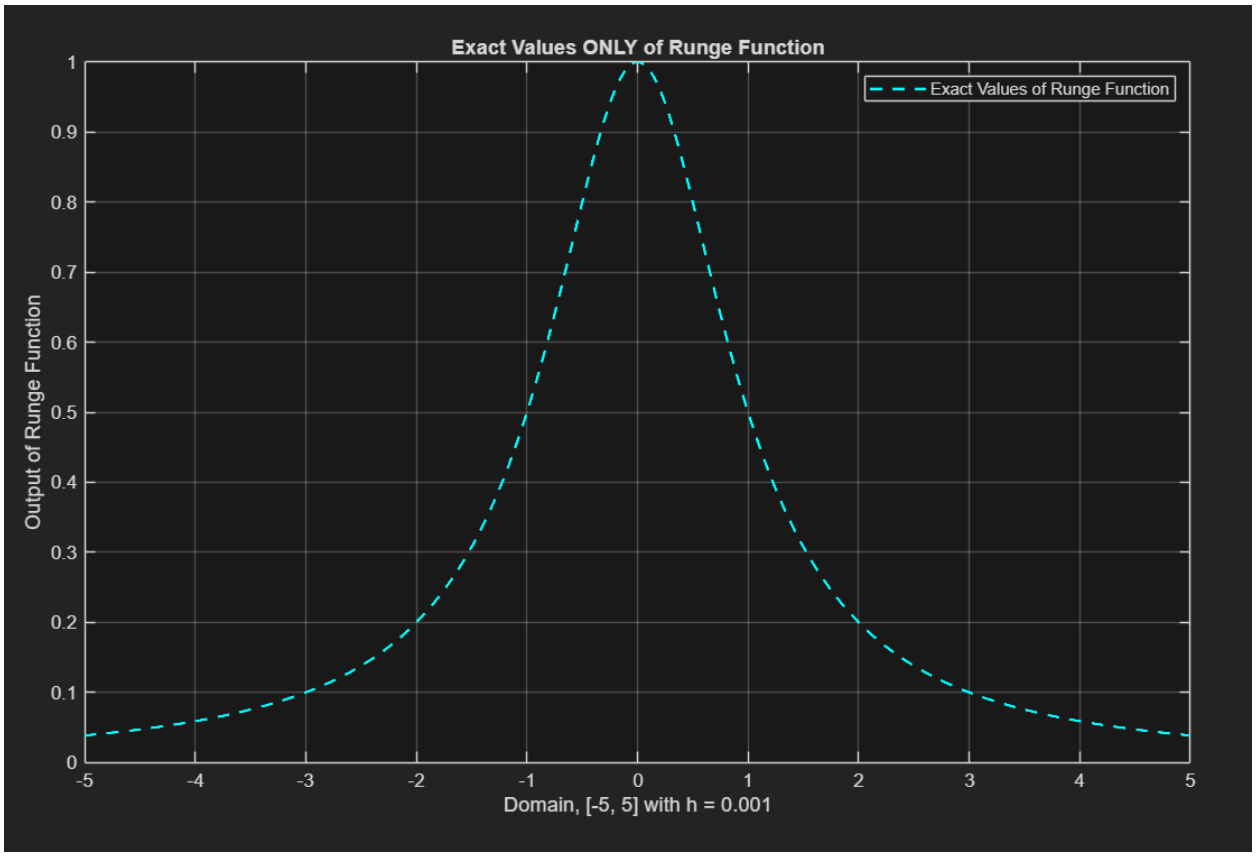
Perform the following tasks."

First, let's (1) code this function to be able to evaluate it more easily, and then (2) plot it to get eyes on it alone. This is a refresh and reuse of the code from Homework 4, but included in this report once again for setup information.

```
%
================================================================
  (1)RUNGE FUNCTION WRITTEN AS FUNCTION IN runge_function.m FILE
%
================================================================

function [y] = runge_function(x)
   y = 1 ./ (1 + (x .^ 2));
end

%
================================================================
  (2)GET EXACT VALUES ON [-5, 5] OF RUNGE AND PLOT IN script.m
  FILE
%
================================================================

% preliminary: get f(x) plotted with a fine mesh
start_pt = -5; end_pt = 5;
h = 0.001; % 10,000 points plotted
x_exact_init = [start_pt:h:end_pt];
y_exact_init = runge_function(x_exact_init);
% plot the exact graph only, mainly for ensuring code is correct
figure('Name', 'Exact Runge');
plot( ...
   x_exact_init, y_exact_init, "--c", ...
   LineWidth=1.5 ...
```

```
   );
hold on;
legend( ...
   'Exact Values of Runge Function' ...
   );
title('Exact Values ONLY of Runge Function');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Output of Runge Function');
grid on;

%
=================================================================
  END CODE FOR COMP PROB 1
%
=================================================================
```

As follows is a plotting of the exact Runge function by the above script.



Now, we will dive into the tasks at hand with the exact function visual in mind.

PART A

"Construct a first degree spline using 21, 41 and 81 equally spaced points as your knots on the interval [−5, 5]. Plot S21(x), S41(x) and S81(x) in a single figure. Report your findings."

In the following code, we will refer to *end_pt* and *start_pt*, but these are simply the same as the exact Runge function above, where *end_pt = 5* and *start_pt = -5*. We will then go through the process of generating equidistant intervals from *n = 21, 41, 81* knots to create degree-1 splines to interpolate the Runge function.

The notes on the assignment sheet say to utilize the *Spline1* code created in class–this is included below as our (1) first step. It has been tidied slightly from the code given, but otherwise remains untouched in the state handed out. Then, we will use it to (2) plot our splines. We will also (3) generate some error plots for analysis.

```
%
================================================================
  (1)SPLINE1 CODE DEVELOPED IN CLASS IN spline_deg_1.m FILE
%
================================================================
function [output] = spline_deg_1(t, y, X)
% t vector represents the knots and y-vec contains y-values
% x is the point to be evaluated by the constructed degree 1
spline
% edited version can take vector of values [a:h:b]
clear output
n = length(t);

for j = 1:length(X)
   x = X(j); % the x we want to evaluate using our splines

   % find the correct interval x is within
   for i = n-1:-1:1
       if (x - t(i)) >= 0;
           break;
       end
   end

   % simple line evaluation given the spline in that interval
```

```
   output(j) = y(i) + (x-t(i)) .* ((y(i + 1) - y(i)) / (t(i + 1)
- t(i)));
end

end

%
==================================================================
  (2)PLOTTING THE SPLINES IN script.m FILE
%
==================================================================

% SPLINE WITH 21 KNOTS

% number of equally spaced nodes
n = 21;  %S21
h = (end_pt - start_pt) / (n - 1); % spacing
t_knots_21 = [start_pt:h:end_pt]; % knot vector
y_table_values_21 = runge_function(t_knots_21); % fill the table
x_finer_21 = [start_pt:(h / 5):end_pt]; %choose a finer grid to
plot points in between knots
% call function, X=evaluation of an interval or point
S_21 = spline_deg_1(t_knots_21, y_table_values_21, x_finer_21);

% ------------------------------------

% SPLINE WITH 41 KNOTS

% number of equally spaced nodes
n = 41;  %S41
h = (end_pt - start_pt) / (n - 1); % spacing
t_knots_41 = [start_pt:h:end_pt]; % knot vector
y_table_values_41 = runge_function(t_knots_41); % fill the table
x_finer_41 = [start_pt:(h / 5):end_pt]; %choose a finer grid to
plot points in between knots
% call function, X=evaluation of an interval or point
S_41 = spline_deg_1(t_knots_41, y_table_values_41, x_finer_41);

% ------------------------------------

% SPLINE WITH 81 KNOTS
```

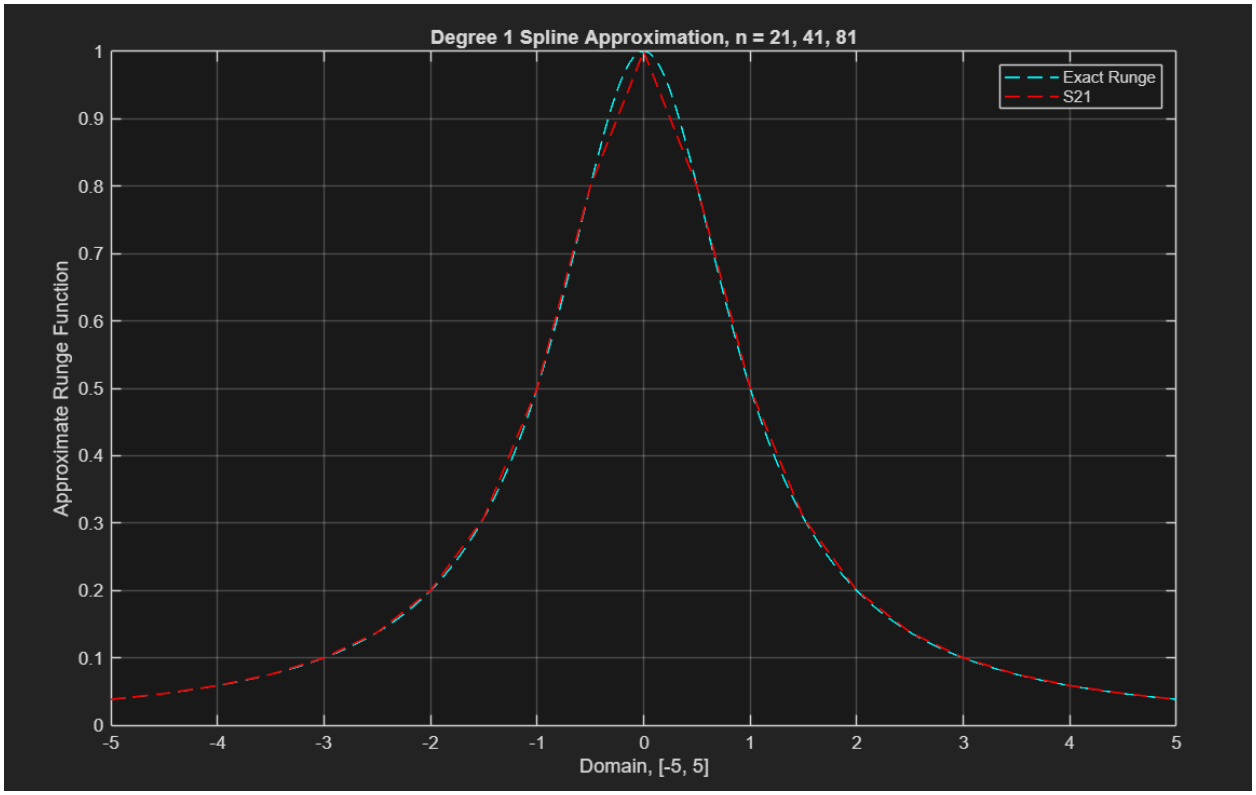
```
% number of equally spaced nodes
n = 81;  %S81
h = (end_pt - start_pt) / (n - 1); % spacing
t_knots_81 = [start_pt:h:end_pt]; % knot vector
y_table_values_81 = runge_function(t_knots_81); % fill the table
x_finer_81 = [start_pt:(h / 5):end_pt]; %choose a finer grid to
plot points in between knots
% call function, X=evaluation of an interval or point
S_81 = spline_deg_1(t_knots_81, y_table_values_81, x_finer_81);

% -------------------------------------

% plot the splines
figure('Name', 'S_21, S_41, S_81');
plot( ...
   ... x_exact_init, y_exact_init, '--c', ...
   x_finer_21, S_21, '--r' , ...
   x_finer_41, S_41, '--y' , ...
   x_finer_81, S_81, '--g',  ...
   ... commented for now, used to confirm intersections:
   ... t_knots_21, y_table_values_21, '*r', ...
   ... t_knots_41, y_table_values_41, '+y', ...
   ... t_knots_81, y_table_values_81, 'og', ...
   LineWidth=1 ...
)
hold on;
legend( ...
   ... 'Exact Runge', ...
   'S21', ...
   'S41', ...
   'S81', ...
   ... uncomment above knot plotting for this to be meaningful:
   ... "S21 Table Vals", "S41 Table Vals", "S81 Table Vals" ...
   LineWidth=1 ...
   )
title('Degree 1 Spline Approximation, n = 21, 41, 81');
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Approximate Runge Function');
grid on;
```

```
%
================================================================
  (3)PLOTTING ABSOLUTE ERROR OF SPLINES IN script.m FILE
%
================================================================

% getting and plotting error for analysis

y_exact_21 = runge_function(x_finer_21);
error_21 = abs(y_exact_21 - S_21);

y_exact_41 = runge_function(x_finer_41);
error_41 = abs(y_exact_41 - S_41);

y_exact_81 = runge_function(x_finer_81);
error_81 = abs(y_exact_81 - S_81);

figure('Name', 'Absolute Error of Spline Approximations');
%plotting the error
plot( ...
   x_finer_21, error_21, '.-r', ...
   x_finer_41, error_41, '.-y', ...
   x_finer_81, error_81, '.-g' ...
   );
hold on;
legend( ...
   "Error of S21", ...
   "Error of S41", ...
   "Error of S81" ...
   );
title('Degree 1 Spline Error, n = 21, 41, 81');
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
grid on;

%
================================================================
  END CODE FOR COMP PROB 1
```

```
%
================================================================
```

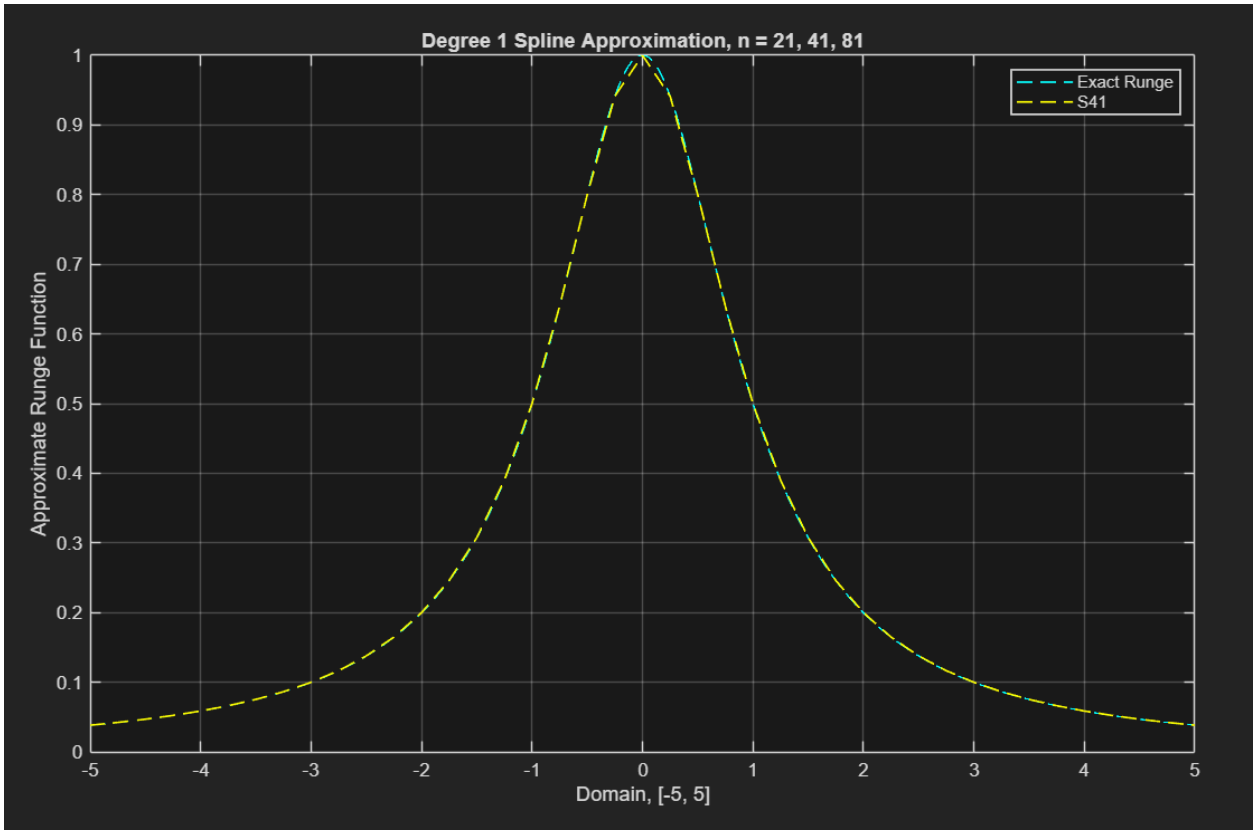
First, let's look at the splines alone against the Runge function to see the visual trend of becoming tighter to the exact function as a clearer visual.

n = 21 VS Runge:



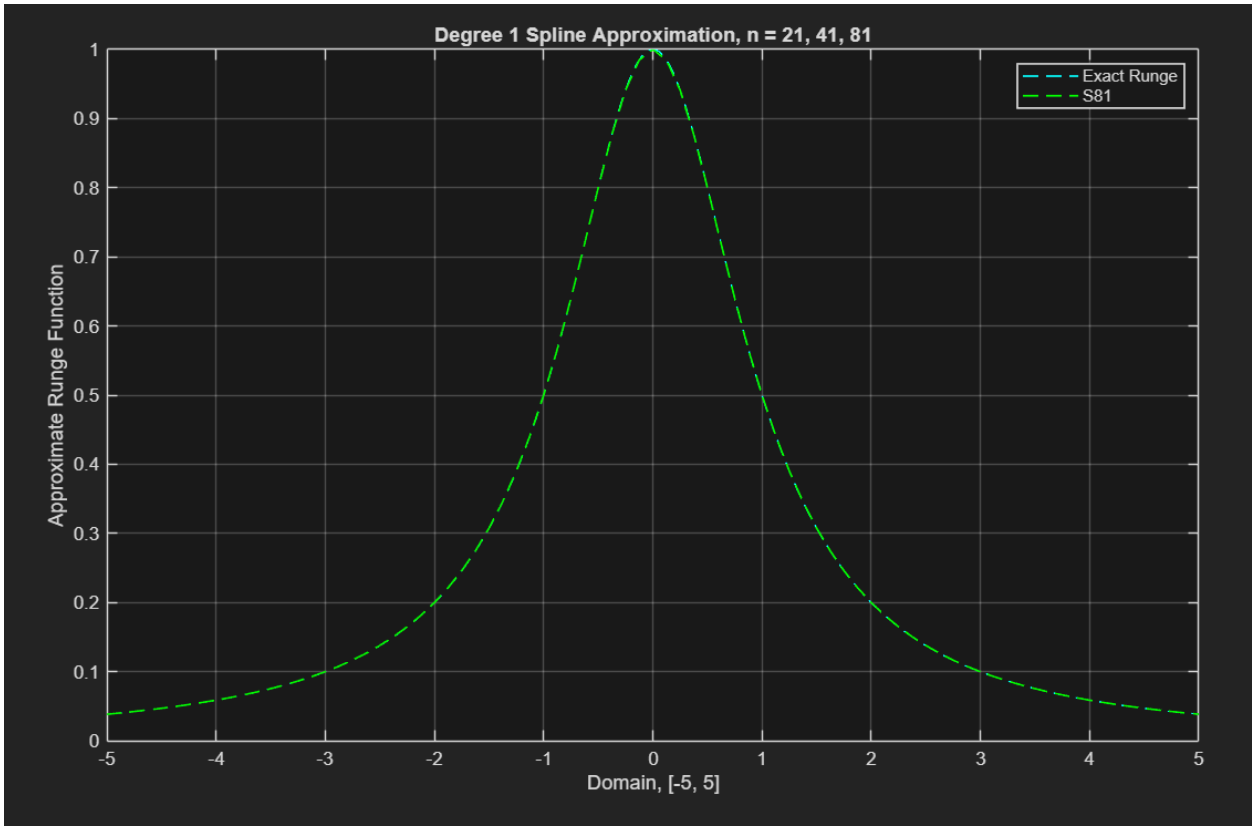
We can see some clear mismatching, especially near the peak where the spline is having trouble matching the curvature of the peak. This makes sense due to the piecewise linearity of degree-1 splines. However, we can do better.

n = 41 VS Runge:

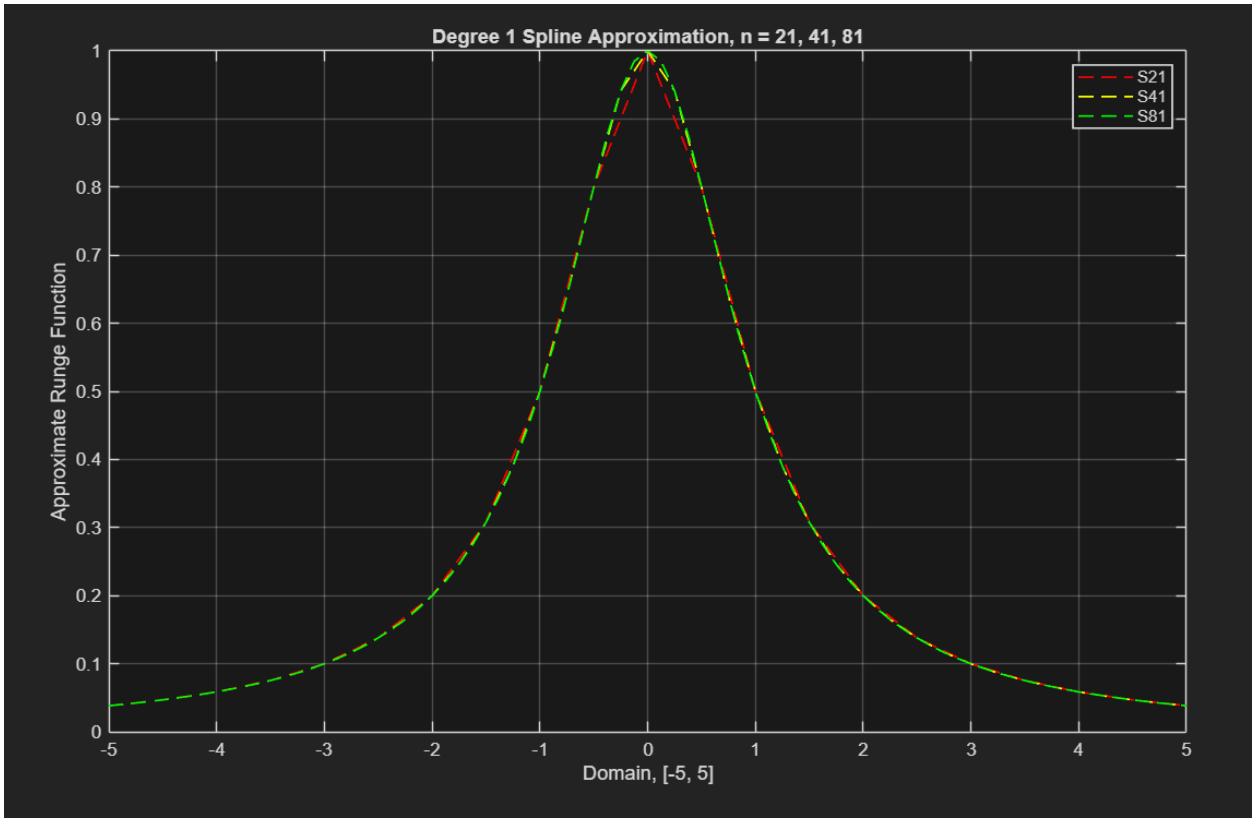


This is clearly a little tighter, especially around the peak where there was the most notable gap between the spline and exact. If we go one further, the trend of accuracy continues.

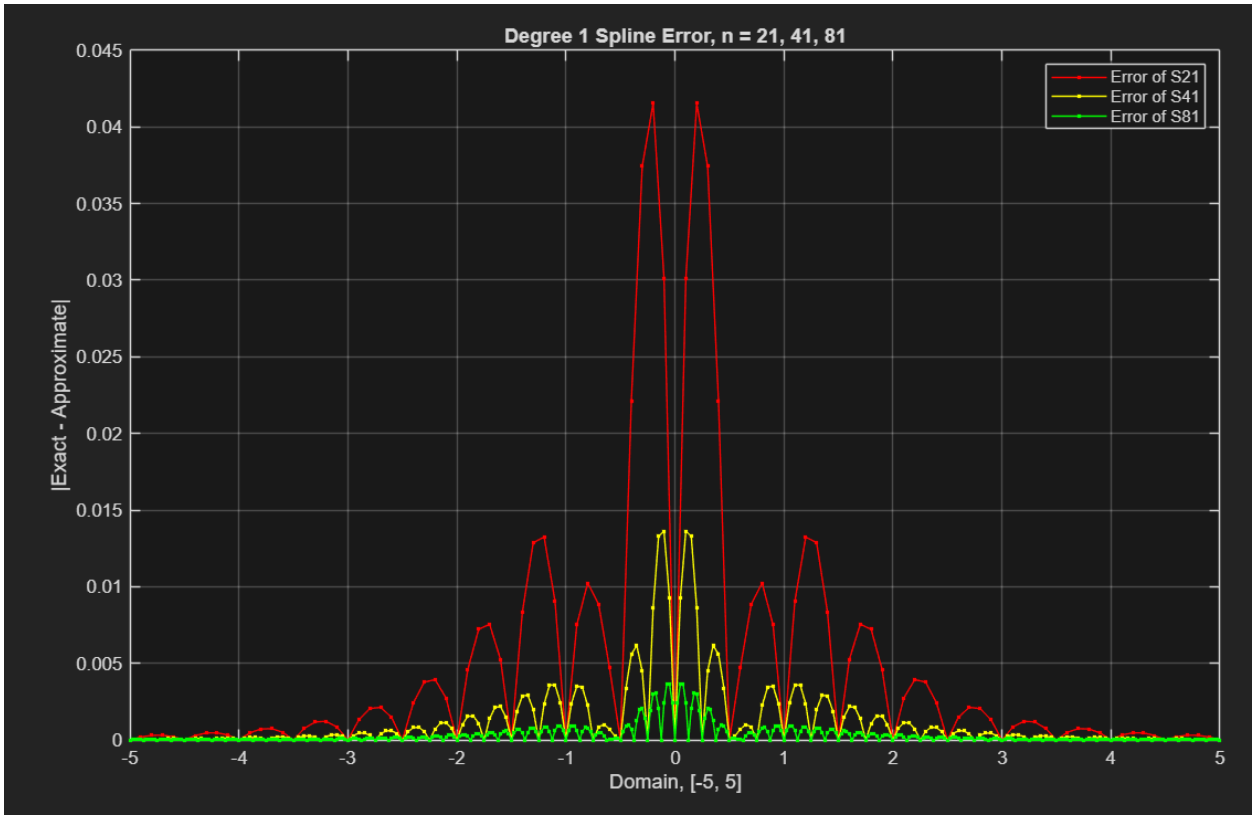
n = 81 VS Runge:



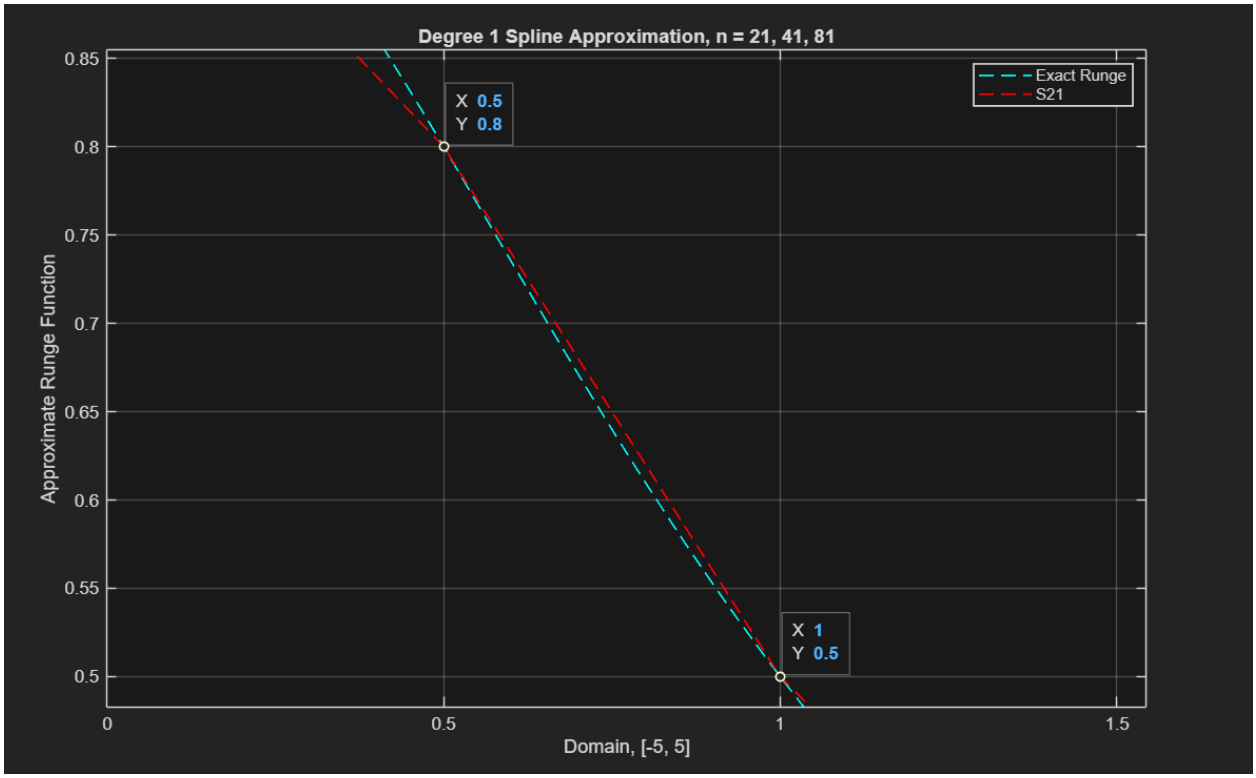
From this visual, there is barely a difference at all, and that curvature has nicely smoothed in our approximation. Now, as the task requested, we can view them all together.



Here we see the gradual smoothing of the spline approximations with increased knots, and therefore smaller intervals upon which to build the piecewise spline. This is highly indicative of the error bound being majorly controlled by the interval size itself–the smaller the intervals, the closer the approximation gets to the exact values. To see this more clearly, observe the error plotting below.



We can note a couple things here. First, the tightness increase with increased knots/smaller intervals is supported by the error visual: the error of *S81* is remarkably smaller than *S21*, being a whole order less than the other at the highest error point. We can also notice, secondly, that the highest interval of error is right around the middle, which is exactly the rounded middle that we notice the lesser knot splines having issues with matching due to the linear nature of a degree-1 spline. Regardless of the addition of more knots compensating for this somewhat well, there is still a notable error peak around this curvature. We also see a third thing from this error plotting: a somewhat “bumpy” pattern to the error plotting. This can be explained again by the linearity of the splines versus the exact, and we can perhaps best see this on *S21*.



Zooming in to an arbitrary interval, we see 2 intersections (or interpolation points) of the exact Runge function and the *S21* spline. We see a clear widening gap as we move away from $x = 0.5$ toward $x = 1$, and then narrowing again the closer we get to $x = 1$, peaking in gap distance right around the mid-way point between points. When we see this gap, we can clearly correspond this to the "bump" in the error plotting from $x = 0.5$ to $x = 1$ to this phenomenon. The degree-1 spline is practically just a ton of lines that cannot bend to match this curve of a non-linear function. So, the more curvature the function has at different points, the higher the error will concentrate in that area; likewise, the more linear the function is at points, the better the degree-1 spline will hold as an approximation to the function.

PART B

"Use 21 ChebyShev nodes and label your polynomial as G20(x). Using those same Cheby-Shev nodes are your knots, construct a 1st degree spline ˜S21(x). Plot the newly formed 1st degree spline and G20(x). Report your findings."

First, we will (1 and 2) reuse the code from Homework 4, Coef and Eval, to generate an interpolating polynomial. Further, we will also (3) reuse the code written for Homework 4 for generating Cheby-Shev nodes on a generic interval. Once we have those setup, we will simply (4) generate the interpolating polynomial *G20* and the degree-1 spline *S21* using Cheby-Shev nodes normally for *G20* and as knots for *S21* and plot the error against exact Runge values.

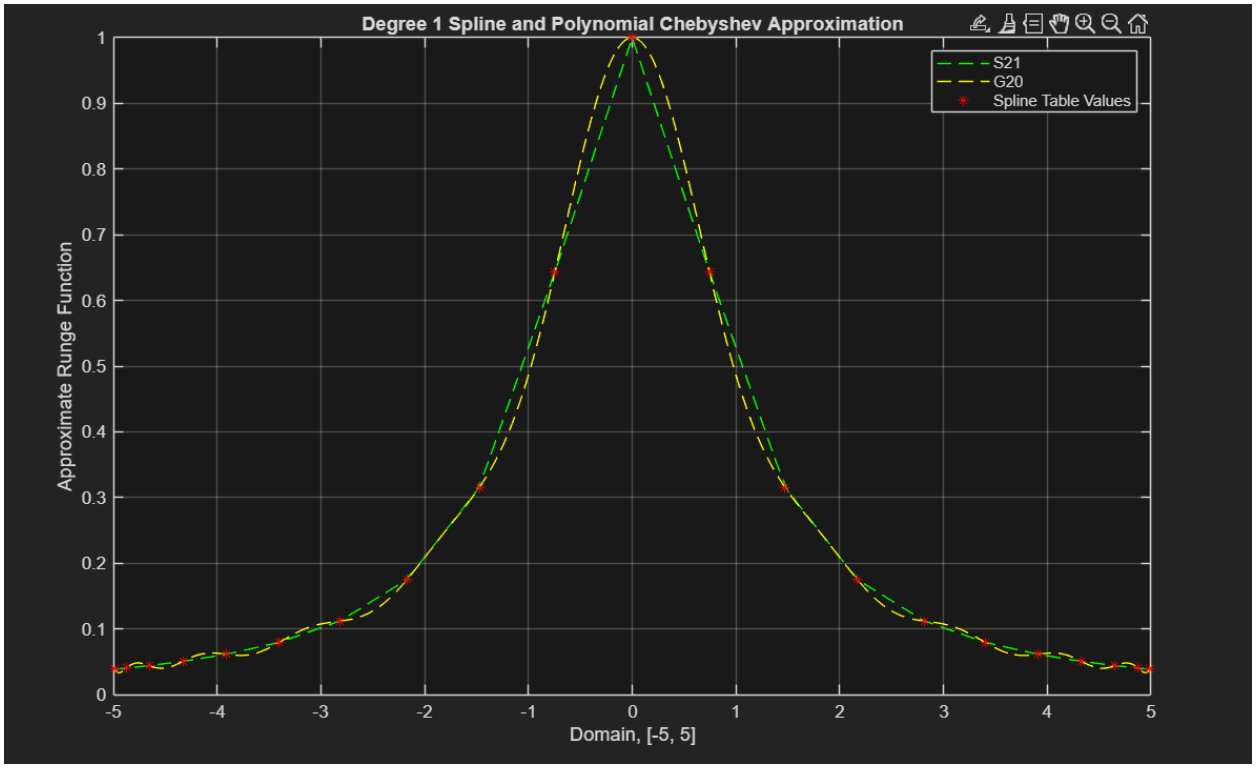
```
%
=================================================================
  (1)COEF PROVIDED AS FUNCTION IN Coef.m FILE
%
=================================================================

function [a] = Coef(n,x,y)
%Coefficients for Interpolation
clear a;
for i=0:n
   a(i+1)=y(i+1);
end
for j=1:n
   for i=n:-1:j
       a(i+1)=(a(i+1)-a(i+1-1))/(x(i+1)-x(i+1-j));
   end
end
end

%
=================================================================
  (2)EVAL PROVIDED AS FUNCTION IN Eval.m FILE
%
=================================================================

function [value] = Eval(n,x,a,t)
%function evaluation - via interpolation
temp=a(end);
for i=n-1:-1:0
   temp=(temp)*(t-x(i+1))+a(i+1);
end
value=temp;
end

%
=================================================================
  (3)GENERATION OF CHEBYSHEV NODES IN chebyshev_nodes.m FILE
%
=================================================================
```

```
function [x_GN] = chebyshev_nodes(num_nodes, start_pt, end_pt)
   x_k = @(k) cos((((2 .* k) + 1) .* pi) ./ (2 .* num_nodes));
   x_k_tilde = @(k) ((start_pt + end_pt) ./ 2) + (((end_pt -
start_pt) ./ 2) * x_k(k));

   for k = 0:(num_nodes - 1);
       x_GN(k + 1) = x_k_tilde(k);
   end;
end

%
==========================================================
  (4)GENERATION OF SPLINE WITH CHEBYSHEV AND INTERPOLATION WITH
  CHEBYSHEV chebyshev_nodes.m FILE
%
==========================================================

% SETUP NODE/KNOT VALUES

n_interp = 20; % G20
% n_interp = 40; % for toying: G40
% n_interp = 60; % for toying: G60
% n_interp = 80; % for toying: G80
n_spline = n_interp + 1; % S(n + 1)

% ------------------------------------

% CONSTRUCT G20

h = 0.01; % arbitrarily chosen mesh
x_G20 = chebyshev_nodes(n_interp + 1, start_pt, end_pt);
y_G20 = runge_function(x_G20); % evaluate data points
a_coeff = Coef(n_interp, x_G20, y_G20); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_G20_Interp = [start_pt:h:end_pt];
for i = 1:length(x_G20_Interp);
   y_G20_Interp(i) = Eval(n_interp, x_G20, a_coeff,
x_G20_Interp(i));
end;

% ------------------------------------
```

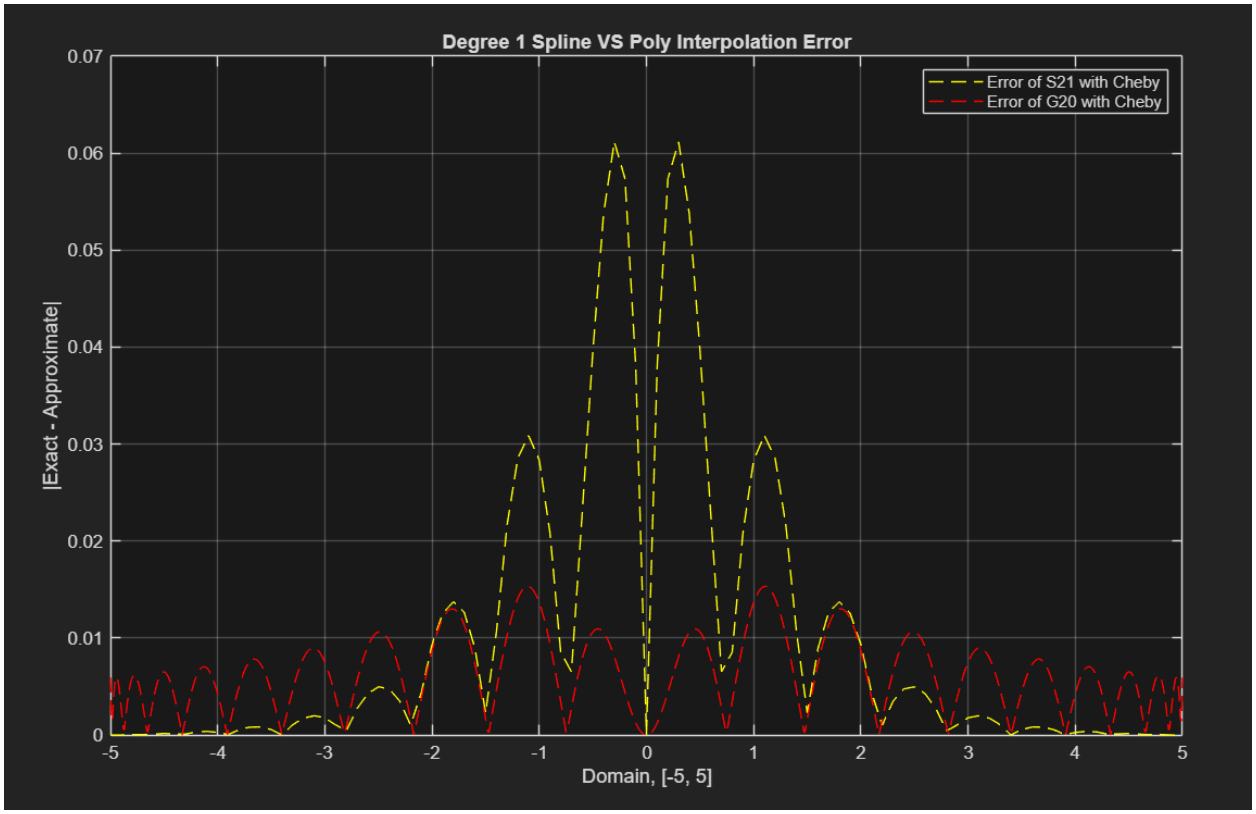
```
% SPLINE WITH 21 CHEBY KNOTS

h = (end_pt - start_pt) / (n_spline - 1); % spacing
t_knots_cheby = flip(x_G20, 2); % knot vector, chebyshev (func
returns descending, reverse return)
y_table_values_cheby = runge_function(t_knots_cheby); % fill the
table
x_finer_21 = [start_pt:(h / 5):end_pt]; %choose a finer grid to
plot points in between knots
% call function, X=evaluation of an interval or point
S_21_cheby = spline_deg_1(t_knots_cheby, y_table_values_cheby,
x_finer_21);

% -------------------------------------

% plot the spline and G20

figure('Name', "S_" + n_spline + ", G_" + n_interp);
plot( ...
   x_finer_21, S_21_cheby, '--g' , ...
   x_G20_Interp, y_G20_Interp, '--y', ...
   t_knots_cheby, y_table_values_cheby, '*r' ...
   )
legend("S" + n_spline, "G" + n_interp, "Spline Table Values");
title('Degree 1 Spline and Polynomial Chebyshev Approximation')
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Approximate Runge Function');
ylim([0, 1]); % limit specified as needed
grid on;

% -------------------------------------

% error plotting

y_exact_cheby_interp = runge_function(x_G20_Interp);
error_cheby_interp = abs(y_exact_cheby_interp - y_G20_Interp);
y_exact_cheby_spline = runge_function(x_finer_21);
error_cheby_spline = abs(y_exact_cheby_spline - S_21_cheby);
figure('Name', 'Absolute Error of Spline/Interp
Approximations'); %plotting the error
```

```
plot( ...
   x_finer_21, error_cheby_spline, '--y', ...
   x_G20_Interp, error_cheby_interp, '--r' ...
   );
hold on;
legend( ...
   "Error of S" + n_spline + " with Cheby", ...
   "Error of G" + n_interp  + " with Cheby"...
   );
title('Degree 1 Spline VS Poly Interpolation Error');
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
% ylim([-0.5, 1]); % limit specified as needed
grid on;

%
=================================================================
  END CODE FOR COMP PROB 1
%
=================================================================
```

Let's look initially at the *G20* and *S21* plots below using Cheby-Shev nodes/knots.



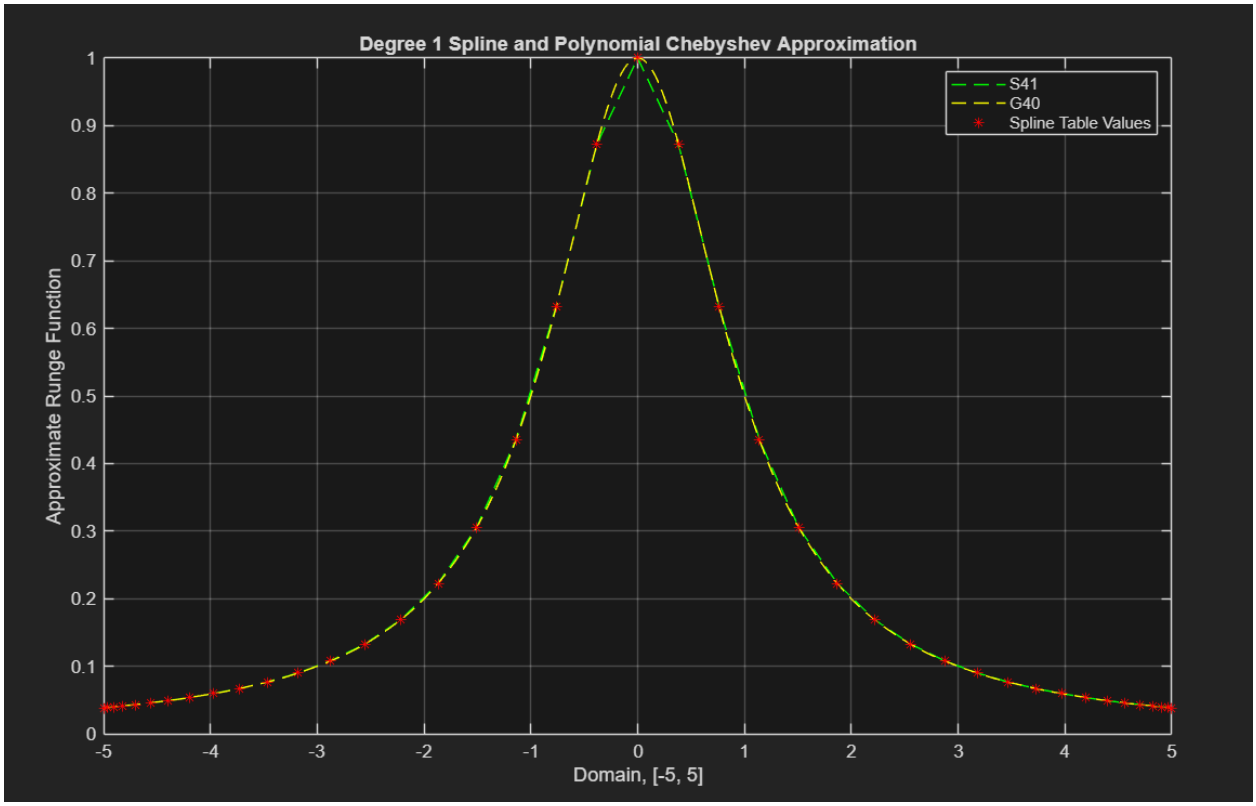
Something to note is the obvious benefit to the curvature of the interpolating polynomial. That clearly fits the Runge function behavior better than the linear *S21* (despite the ever so slight oscillations at the edges). Further, let's look at the absolute error of the two interpolations.



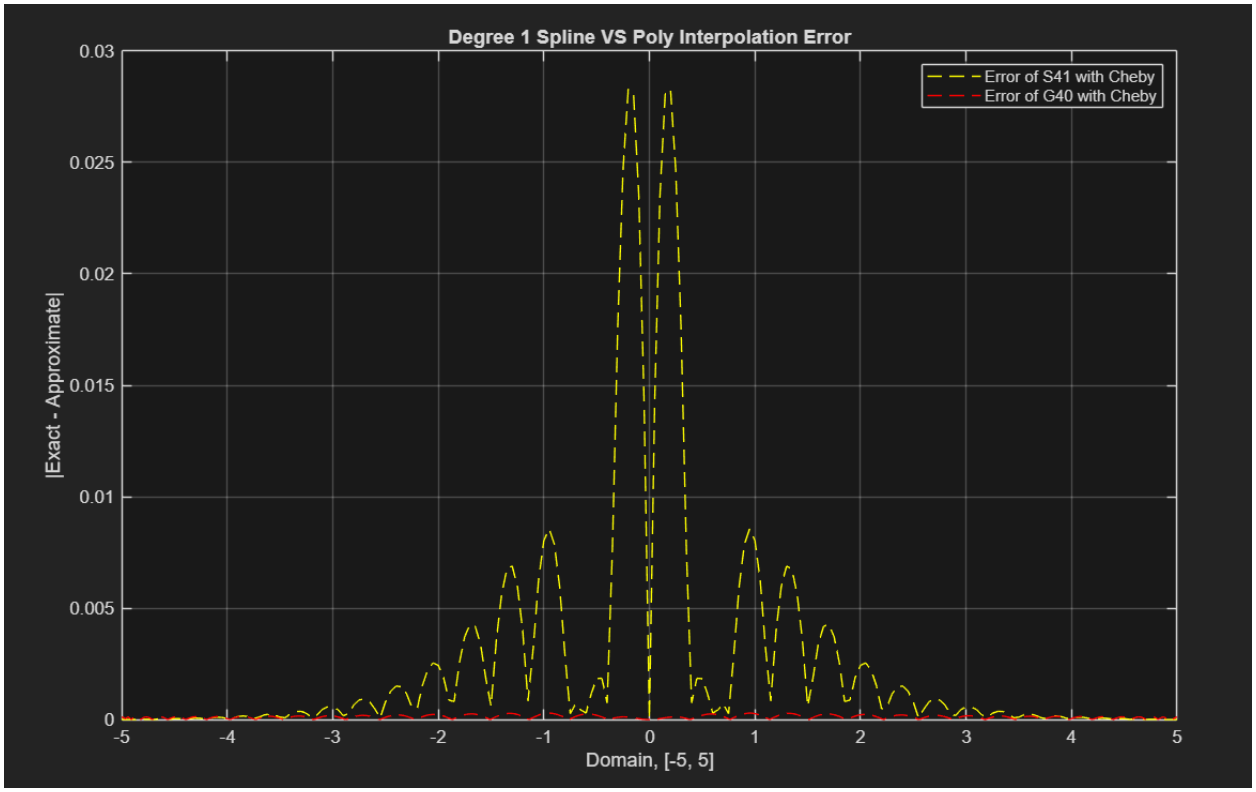
We can see from above some interesting things to note. On *S21*, the error peaks distinctly higher than before when using equispaced knots. The prior peak around the center curve was *~0.04* as seen in prior plotting, so *0.06* is a notable increase considering all we changed where the knots used (and, really, the intervals). Further, we can see that the error of *S21* still tapers off as expected as Runge becomes more linear near the edges. Despite the expected taper off, the jump in the middle is evidence of the error of the first degree spline being directly tied to the interval length. Chevy-Shev nodes are specifically not equally spaced, which is partially why the wild oscillations of the polynomial interpolation are subdued when using these points as nodes–more nodes where the oscillations occur give more foundationally accurate polynomials, as we saw in Homework 4. This does mean an effect on the spline though: since Cheby-Shev nodes are farther apart around $x = 0$, the intervals around there are longer than the middle, thereby increasing the error of the spline around $x = 0$, or the peak curve of Runge. So, there are a few factors that affect the error of this spline, and Cheby-Shev has an adverse effect on that interpolation.

To note as well, *G20* also has a large decrease in error around the curve due to its polynomial-curve nature. Regardless, the error is higher around the edges, faring worse than the spline due to the slight oscillations of the polynomial. Let's try adding nodes to help that:

S41, G40 with Cheby-Shev Nodes/Knots

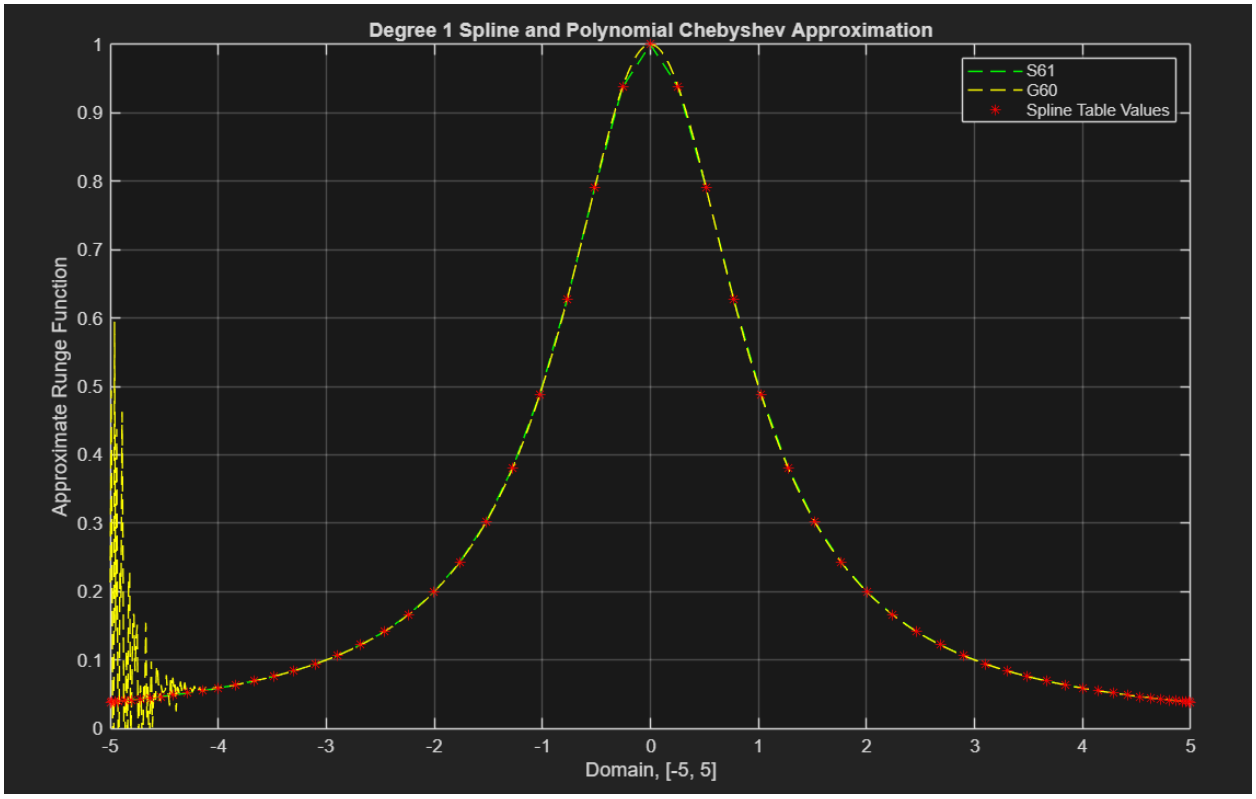


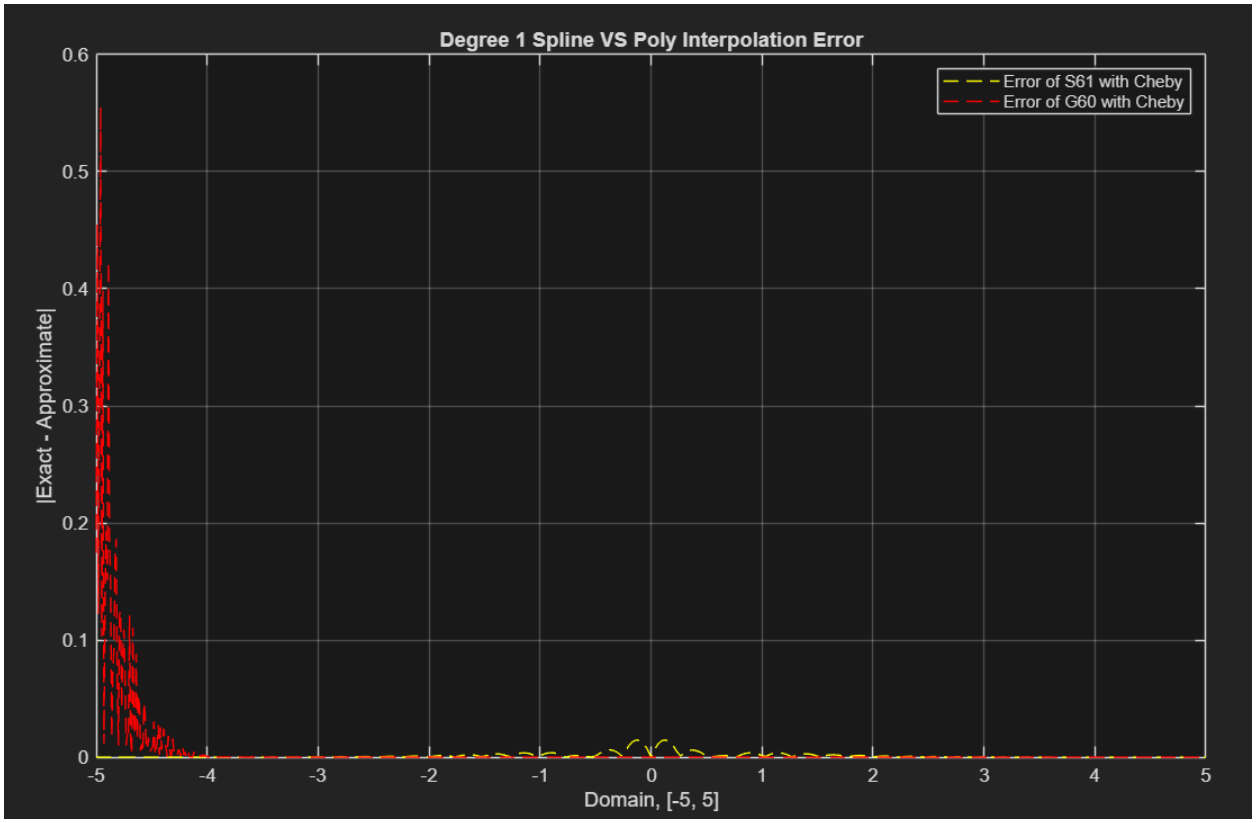
Both above look tighter to what we expect, with the exception of *S41* still having issues around that long-interval peak. To glance at the error, we have supporting results.



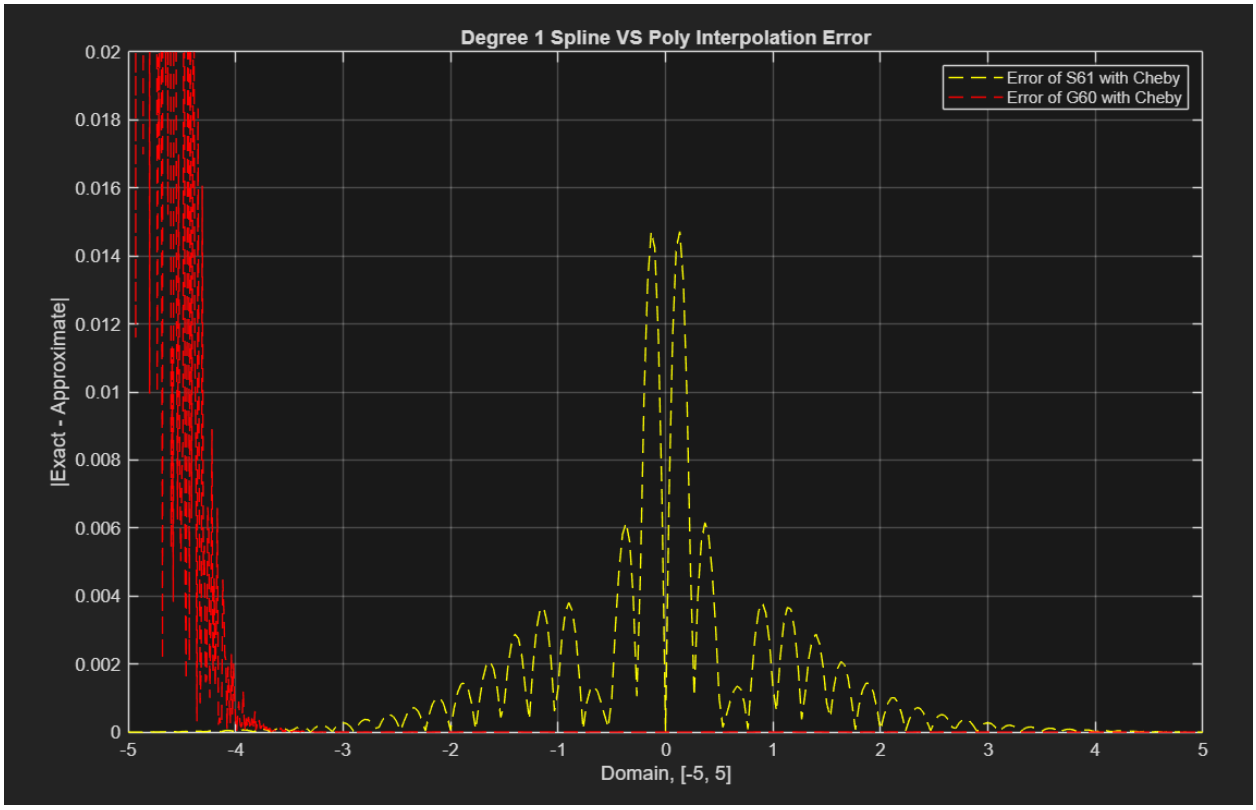
The spline, despite having a *much* higher error than the now highly accurate polynomial, has still lowered in error even with the known error sources. We will try one more test: we know from Homework 4 that the *G60* polynomial with Cheby-Shev nodes starts to wildly oscillate, meaning that adding more Cheby-Shev nodes does not necessarily mean a more accurate polynomial, and that this technique is beneficial up to a point due to peculiarities with the Runge function/phenomenon. So, let's see how the spline fares comparatively when using *60* and *80* nodes.

S61, G60 with Cheby-Shev Nodes/Knots (the known fail point)


Degree 1 Spline and Polynomial Chebyshev Approximation
S61
G60
Spline Table Values
Approximate Runge Function
Domain, [-5, 5]

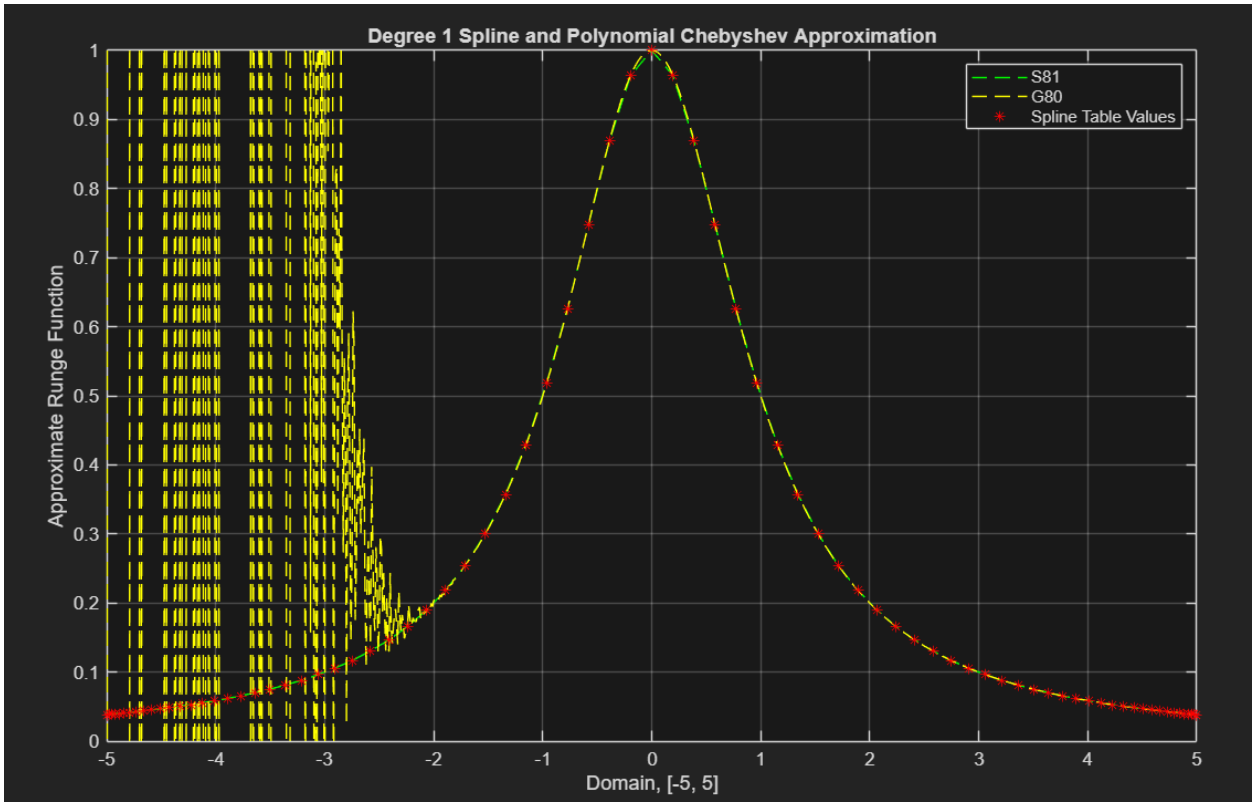

Degree 1 Spline VS Poly Interpolation Error
Error of S61 with Cheby
Error of G60 with Cheby
|Exact - Approximate|
Domain, [-5, 5]

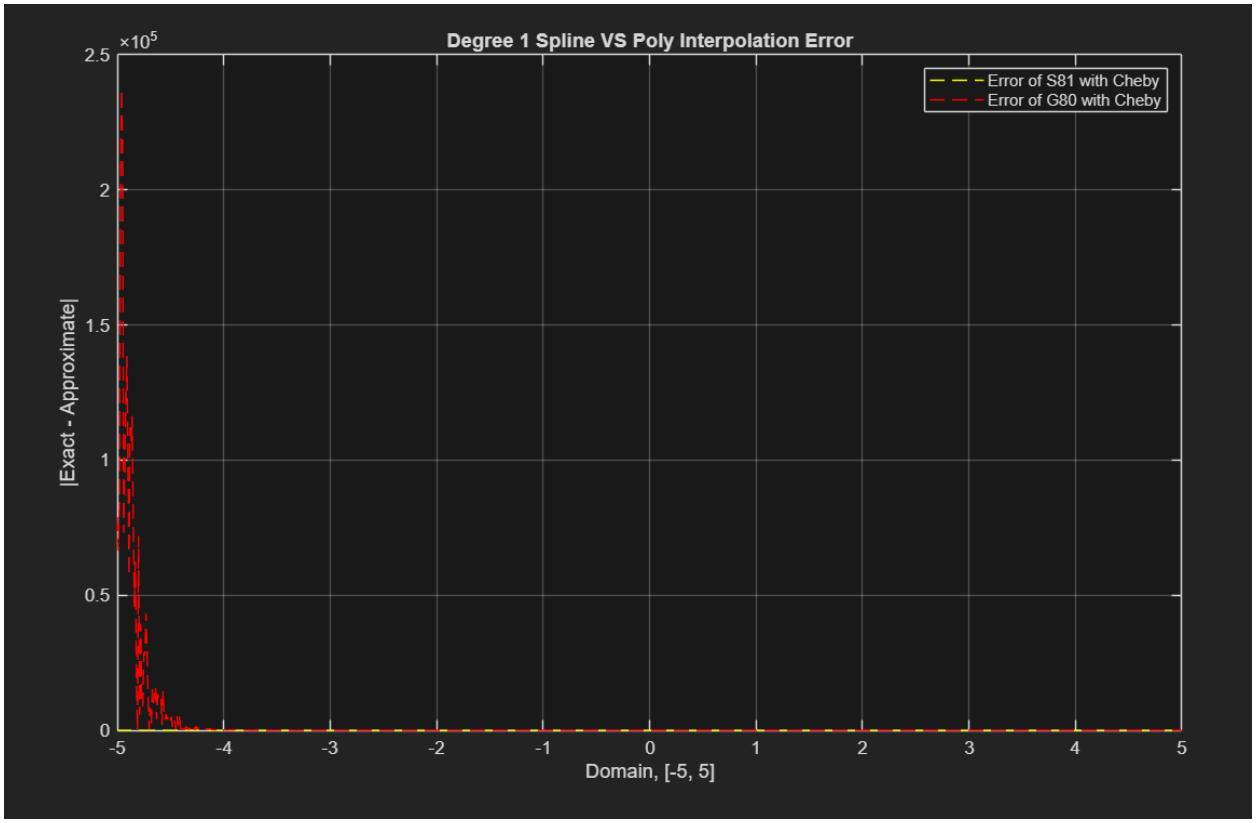
To zoom in on the center to see how the two compare, let's limit the range to *[0, 0.02]*.



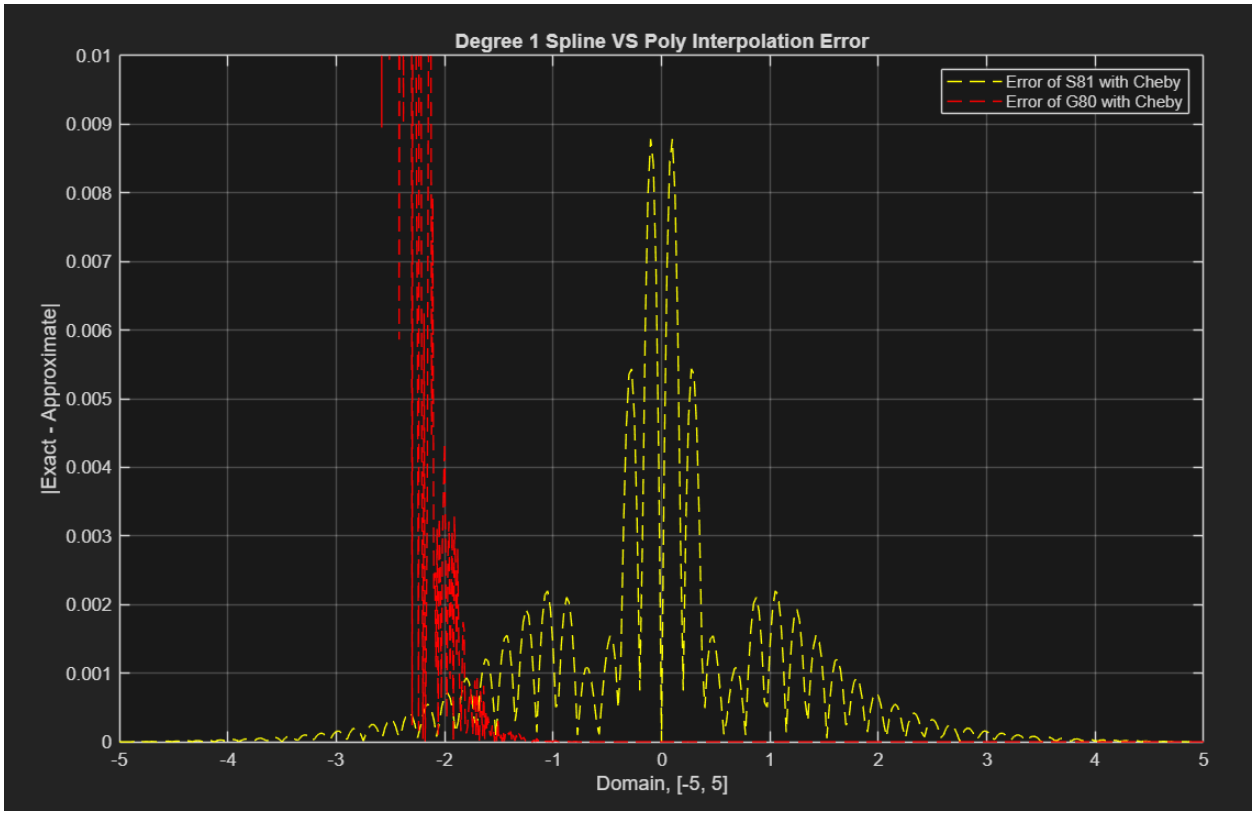
And now, let's continue to see the trend.

S81, G80 with Cheby-Shev Nodes/Knots


Degree 1 Spline and Polynomial Chebyshev Approximation
S81
G80
Spline Table Values
Approximate Runge Function
Domain, [-5, 5]


Degree 1 Spline VS Poly Interpolation Error
×10^5
Error of S81 with Cheby
Error of G80 with Cheby
|Exact - Approximate|
Domain, [-5, 5]

Again, let's impose a limit on the range of *[0, 0.01]* to zoom in on the center problem area.



What we can see above is an explosion in error of the polynomial while the spline continues the downward trend to *infinitesimal error from the exact function values*. Despite the spline still technically doing worse than the polynomial around the center and on values, that oscillation is growing in both range and domain, rendering the polynomial completely useless. So, even with the handicaps of curvature and non-equidistant intervals, the spline successfully interpolates the Runge function, curbing the oscillation phenomenon, where the polynomial inevitably fails.

CONCLUSION

There are multiple ways to approximate and interpolate a function. Creating unique Lagrange and Newton form polynomials to approximate complicated functions/match values in a table can be a very effective tool to utilize. We can even get a high degree of accuracy in strange situations, like the Runge Phenomenon, by cleverly selecting the values of a function to use to minimize unfortunate oscillations. Regardless, there are certain situations in which, if not used wisely, the polynomial will fail catastrophically.

Enter the naive degree-1 spline. It is a remarkably simple way to generate an interpolation of table values or approximation to a complex function. Its beauty is in the simplicity of it, but it is not without its pitfalls and considerations. However, compared to a polynomial, it behaves wonderfully concretely, with no surprise error sources.

Both techniques are valid ways to approximate data points. It is consequently our job to analyze the situation and understand which is more appropriate given a function, the data, or whatever variables there may be in the problem we are to solve.